Course Outline: Object-Relational Mapping: The Universal Translator

Title: Object-Relational Mapping: The Universal Translator **Skill:** Skill 33 — Relacionando objetos y tablas utilizando un ORM **Learnpack:** + Object-Relational Mapping: El traductor universal **File:** s33-w12d34-object-relational-mapping-el-traductor-u **Prerequisite:** Representación de objetos en Python (s32-w11d33-representacion-de-objetos-en-python) **Target Audience:** Beginner developers who know Python classes and SQL fundamentals, now ready to connect both worlds **Total Lessons:** 12 **Format:** Mixed (17% hands-on = 2 lessons)

Course Description

You know how to write Python classes. You know how to write SQL queries. Until now, they've lived in separate worlds — Python in your code, SQL in your database. This course introduces the ORM: the layer that translates between them. You'll learn how Python classes map to database tables, how instances map to rows, how relationships between objects mirror foreign keys, and how to talk to the database using a session instead of raw SQL. By the end, you'll be able to define, relate, and query database models entirely in Python — without forgetting that SQL is still running underneath.

Course Philosophy

This course emphasizes:

- Building on what students already know: Python classes (Skill 32) and SQL relationships (Skills 30–31) are the foundation, not prerequisites to re-explain
- The ORM as a translator, not a replacement — students must understand the SQL that ORM generates
- Seeing both sides: every ORM concept is shown alongside its SQL equivalent, so the mapping is explicit

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - The ORM Bridge	3
02 - Relationships	3
03 - Sessions and Transactions	3
04 - Assessment	1
05 - Outro	1

Section	Lessons
Total	**12**

00.0 The Universal Translator 🕮

Learning Objectives:

- Understand the gap this course bridges: Python objects on one side, SQL tables on the other
- Know what an ORM does and why it exists
- Preview the course journey from model definition to querying

Content Outline:

- The two worlds: Python classes that represent data, and SQL tables that store it — and the friction of manually converting between them
- What an ORM does: it maps one to the other, so you write Python and the ORM generates the SQL
- What this course covers: the correspondence (how Python maps to SQL), relationships (1:1, 1:n, n:m), sessions (how you talk to the DB), and best practices
- What this course does NOT do: replace your SQL knowledge — understanding what the ORM generates is part of using it well

Transitions: This lesson sets the stage. Section 01 dives into the core idea: what an ORM is and how the mapping works.

Key Principles:

- An ORM is a translation layer, not a magic box — what runs underneath is always SQL
- Students who know Python classes and SQL are already 80% of the way there

Examples:

- Without ORM: write a `User` class in Python, then write separate SQL to create a `users` table, then write more SQL to insert/query rows — three separate things to keep in sync
- With ORM: define `User` once as a Python class, and the ORM handles the table, the inserts, and the queries

---

01.0 The ORM Bridge 🕮

Learning Objectives:

- Understand what an ORM is at a conceptual level
- Know why ORMs exist and what problem they solve
- Recognize that ORM knowledge requires SQL knowledge — not instead of it

Content Outline:

- The impedance mismatch: relational databases think in tables and rows; Python thinks in objects and instances — every time data moves between them, it must be converted
- What an ORM solves: it automates that conversion, letting you write Python objects and get SQL operations
- The most common Python ORM: SQLAlchemy (used by Flask, FastAPI, and many other frameworks)
- Why ORM doesn't replace SQL: the ORM generates SQL — if you don't understand that SQL, you can't debug slow queries, can't reason about what's happening in the database, and can't catch mistakes
- Anti-pattern — The Black Box: treating the ORM as magic and never checking what SQL it generates. This leads to undetected N+1 queries, incorrect joins, and performance issues that are impossible to diagnose
- Sub-lesson preview: 01.1 shows the exact mapping — class to table, attribute to column, instance to row

Transitions: This lesson establishes the mental model. 01.1 makes the mapping concrete.

Key Principles:

- ORM = Object-Relational Mapping: one side is objects (Python), the other is relations (SQL tables)
- The ORM is a translator — both languages still exist underneath
- You must understand SQL to use an ORM responsibly

Examples:

- A `User` class in Python ↔ a `users` table in PostgreSQL
- `user = User(name="Ana")` ↔ `INSERT INTO users (name) VALUES ('Ana')`
- `session.query(User).all()` ↔ `SELECT * FROM users`

01.1 The Correspondence 🕮

Learning Objectives:

- Map each ORM concept to its SQL equivalent: class → table, attribute → column, instance → row
- Select the correct SQLAlchemy column type for a given data type
- Understand the role of migrations as the mechanism that applies model definitions to the database

Content Outline:

- The mapping table:
  - Python class → SQL table
  - Class attribute (Column) → table column
  - Object instance → table row
  - Primary key attribute → primary key column
- Column types in SQLAlchemy and their SQL equivalents: `Integer`, `String(n)`, `Boolean`, `Float`, `DateTime`, `Text`

- `__tablename__`: how the class tells SQLAlchemy which table it maps to
- `declarative_base()`: the foundation all model classes inherit from
- Migrations — the bridge between model and database: defining a model in Python does not automatically create the table. A migration is the process that reads your model and applies the change to the actual database schema. (Deep dive on Alembic and FastAPI migrations is the next learnpack.)

Transitions: This lesson covers the one-class, one-table scenario. 01.2 is the hands-on exercise to write one.

Key Principles:

- Every attribute in an ORM model corresponds to exactly one column in the database
- The Python class is the blueprint; the database table is what gets stored
- Migrations are the deployment step for your schema — they make the class definition real in the database

Examples:

```
from sqlalchemy import Column, Integer, String, Boolean
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = "users"          # → creates table named "users"

    id = Column(Integer, primary_key=True)    # → id INTEGER PRIMARY KEY
    username = Column(String(80))             # → username VARCHAR(80)
    is_active = Column(Boolean, default=True) # → is_active BOOLEAN DEFAULT
TRUE
```

- `User` class → `users` table
- `username = Column(String(80))` → `username VARCHAR(80)` column
- `user = User(username="Ana")` → one row in the `users` table

01.2 Write a Model 💻

Challenge Prompt: Define a SQLAlchemy `User` model with an integer primary key, a username string (max 80 chars), a unique email string (max 120 chars), and a boolean `is_active` attribute defaulting to `True`, then print each column name and type.

Solution Code:

```
from sqlalchemy import Column, Integer, String, Boolean
from sqlalchemy.ext.declarative import declarative_base
```

```
Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    username = Column(String(80), nullable=False)
    email = Column(String(120), nullable=False, unique=True)
    is_active = Column(Boolean, default=True)

    def __repr__(self):
        return f"<User {self.username}>"

for column in User.__table__.columns:
    print(f"{column.name}: {column.type}")
```

Challenge Code:

```
from sqlalchemy import Column, Integer, String, Boolean
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    # Define: id (Integer, primary_key), username (String 80, not null),
    # email (String 120, unique, not null), is_active (Boolean, default True)
    pass

    def __repr__(self):
        return f"<User {self.username}>"

for column in User.__table__.columns:
    print(f"{column.name}: {column.type}")
```

02.0 Relationships 🕮

Learning Objectives:

- Understand how ORM relationships mirror the SQL foreign key relationships already known from Skill 31
- Know the three relationship types (1:1, 1:n, n:m) and how each is expressed in SQLAlchemy
- Recognize what ORM adds on top of foreign keys: the ability to navigate from object to object

Content Outline:

- What you already know: SQL uses foreign keys to link tables. A `posts` table has a `user_id` column that references `users.id`. You query across them with JOIN.
- What ORM adds: once the foreign key is declared, you can navigate from one object to its related objects directly in Python — no JOIN needed in your code
- The three relationship types and their ORM equivalents:
  - 1:1 → `ForeignKey` + `relationship(uselist=False)`
  - 1:n → `ForeignKey` on the "many" side + `relationship()` on the "one" side
  - n:m → association table + `relationship(secondary=...)`
- Sub-lesson preview: 02.1 covers 1:1 and 1:n with ForeignKey and relationship(); 02.2 covers n:m with association tables

Transitions: This overview sets the mental model. 02.1 goes deep on 1:1 and 1:n.

Key Principles:

- ORM relationships do not replace foreign keys — they layer on top of them
- The foreign key is what the database enforces; the `relationship()` is what makes navigation convenient in Python
- You still need to understand JOIN to understand what the ORM generates when you access a relationship

Examples:

- SQL: `posts.user_id REFERENCES users.id` → in Python: `post.author` gives you the `User` object directly
- What was a JOIN in SQL becomes dot notation on an object

02.1 One-to-One and One-to-Many 🕮

Learning Objectives:

- Declare a foreign key in SQLAlchemy using `ForeignKey`
- Define a `relationship()` on a model to enable object-level navigation
- Distinguish between 1:1 (`uselist=False`) and 1:n (default) relationships
- Use `back_populates` to make the relationship navigable from both sides

Content Outline:

- `ForeignKey`: declares the foreign key column on the "many" or "child" side — equivalent to `REFERENCES` in SQL
- `relationship()`: defines the Python-level link, placed on the model you want to navigate FROM
- 1:n example: `User` has many `Post` objects — `Post` holds the `ForeignKey("users.id")`, `User` holds `relationship("Post", back_populates="author")`
- `back_populates`: wires both sides — `post.author` gives you the `User`, `user.posts` gives you all their `Post` objects
- 1:1 variant: add `uselist=False` to the `relationship()` — the result is a single object instead of a list

- What happens in SQL: accessing `user.posts` triggers a `SELECT * FROM posts WHERE user_id = ?` — the ORM generates the query

Transitions: 1:1 and 1:n share the same foreign key + relationship pattern. 02.2 covers the more complex n:m case.

Key Principles:

- The foreign key always lives on the "many" side (the child)
- `relationship()` is a Python convenience — the database only knows about the foreign key
- `back_populates` must be set on both sides for bidirectional navigation to work

Examples:

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String(80))
    posts = relationship("Post", back_populates="author")  # 1:n

class Post(Base):
    __tablename__ = "posts"
    id = Column(Integer, primary_key=True)
    title = Column(String(200))
    user_id = Column(Integer, ForeignKey("users.id"))      # FK on child
    author = relationship("User", back_populates="posts")

# Navigation (no JOIN needed in Python code):
# user.posts  → list of Post objects
# post.author → User object
```

02.2 Many-to-Many 🕮

Learning Objectives:

- Explain why n:m relationships require an association (pivot) table
- Define an association table using SQLAlchemy's `Table` object
- Declare a `relationship()` with the `secondary` parameter to connect two models through an association table

Content Outline:

- Why n:m needs a pivot table: a student can enroll in many courses; a course can have many students — neither table can hold a foreign key to the other directly

- The association table: a plain `Table` object (not a full model class) with two foreign key columns — one to each related table. Equivalent to what you wrote in SQL as a pivot table in Skill 31.
- `secondary=`: the `relationship()` parameter that points to the association table — SQLAlchemy uses it to generate the correct JOINs
- Anti-pattern — Skipping the Association Table: some beginners put a comma-separated string in a column to represent multiple relationships. This destroys data integrity, makes querying impossible with SQL, and cannot be enforced with foreign keys. Always use a proper pivot table.
- When to use a full association model instead: if the pivot table needs its own attributes (e.g., enrollment date), it becomes a full model class, and you define two 1:n relationships instead

Transitions: This completes the three relationship types. Section 03 shifts from defining the schema to actually using it — talking to the database through a session.

Key Principles:

- n:m always requires a third table — this is true in SQL and in ORM
- The `secondary` table is an invisible intermediary — SQLAlchemy handles the JOINs automatically
- If the association itself has data, model it as a full class with two 1:n relationships

Examples:

```
from sqlalchemy import Table, Column, Integer, ForeignKey
from sqlalchemy.orm import relationship

# Association table (plain Table, not a model class)
enrollment = Table(
    "enrollment",
    Base.metadata,
    Column("student_id", Integer, ForeignKey("students.id")),
    Column("course_id", Integer, ForeignKey("courses.id")),
)

class Student(Base):
    __tablename__ = "students"
    id = Column(Integer, primary_key=True)
    name = Column(String(80))
    courses = relationship("Course", secondary=enrollment,
back_populates="students")

class Course(Base):
    __tablename__ = "courses"
    id = Column(Integer, primary_key=True)
    title = Column(String(200))
    students = relationship("Student", secondary=enrollment,
back_populates="courses")

# student.courses → list of Course objects
# course.students → list of Student objects
```

03.0 Sessions and Transactions 🕮

Learning Objectives:

- Understand what a session is and why it exists as an intermediary between Python and the database
- Know the lifecycle of a session: open, add, commit or rollback, close
- Understand what a transaction is and why atomicity matters

Content Outline:

- The session as the gateway: every interaction with the database goes through a session object — you don't send SQL directly anymore
- What a session does: it tracks changes to your Python objects and knows which ones need to be saved, updated, or deleted in the database
- What a transaction is: a group of database operations that either all succeed or all fail together — the database is never left in a half-changed state
- `commit`: finalizes all pending changes and writes them to the database permanently
- `rollback`: cancels all pending changes and restores the session to its last committed state
- Sub-lesson preview: 03.1 covers the actual session operations (add, query, commit, rollback) and common pitfalls

Transitions: Sections 01 and 02 defined the schema. This section is about using it — adding records, querying them, and handling errors correctly.

Key Principles:

- Nothing is saved to the database until you call `commit()`
- A session tracks your objects like a shopping cart — `commit()` is checkout
- If something goes wrong, `rollback()` empties the cart without affecting the database

Examples:

- `session.add(user)` → stages the new User for insertion (not saved yet)
- `session.commit()` → executes `INSERT INTO users ...` and saves permanently
- `session.rollback()` → discards staged changes, database unchanged

03.1 Working with Sessions 🕮

Learning Objectives:

- Add single and multiple objects to a session using `add()` and `add_all()`
- Query the database using `session.query()` with filters
- Commit changes and handle rollbacks on error
- Identify the N+1 query problem and explain how eager loading solves it
- Apply transaction boundaries correctly: keep transactions short and scoped

Content Outline:

- Creating and closing a session: `Session = sessionmaker(bind=engine)` → `session = Session()`
- Adding objects: `session.add(obj)` stages one object; `session.add_all([obj1, obj2])` stages multiple
- Querying: `session.query(User).all()` → all rows; `.filter_by(username="ana")` → filtered; `.first()` → single result
- `commit()` and `rollback()` in error handling: wrap operations in try/except, call `rollback()` in the except block to avoid leaving the session in a broken state
- The N+1 problem: if you load 100 posts and then access `post.author` in a loop, the ORM executes 1 query for the posts plus 100 more for each author — 101 queries total. This is a silent performance killer.
- Fixing N+1 with eager loading: `session.query(Post).options(joinedload(Post.author)).all()` — one query with a JOIN instead of 101 separate queries
- Lazy loading (default): related objects are fetched only when accessed — fine for small data, dangerous at scale
- Transaction best practices: keep each commit to one logical operation; never leave a session open across a web request boundary; use `try/commit/except/rollback/finally/close` for safety

Transitions: This lesson covers everything you need to work with sessions in practice. 03.2 is the exercise to apply it.

Key Principles:

- Query results are Python objects — you navigate them with dot notation, not SQL
- N+1 is invisible until it's a performance crisis — always check what queries the ORM generates in production
- `rollback()` is not optional in error handling — an uncommitted, failed session is a broken session

Examples:

```
# Add and commit
user = User(username="ana", email="ana@example.com")
session.add(user)
session.commit()

# Query with filter
user = session.query(User).filter_by(username="ana").first()
print(user.email)

# Error-safe pattern
try:
    session.add(User(username="bob", email="bob@example.com"))
    session.commit()
except Exception:
    session.rollback()
```

```
    raise
finally:
    session.close()

# Eager loading (fixes N+1)
from sqlalchemy.orm import joinedload
posts = session.query(Post).options(joinedload(Post.author)).all()
for post in posts:
    print(post.author.username)  # no extra query per post
```

03.2 Manage Records 💻

Challenge Prompt: Using the provided `Product` model and session setup, add a Product named "Laptop" with price 999 to the session, commit it, then query all products and print each product's name and price.

Solution Code:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

Base = declarative_base()

class Product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    price = Column(Integer, nullable=False)

engine = create_engine("sqlite:///:memory:")
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()

laptop = Product(name="Laptop", price=999)
session.add(laptop)
session.commit()

products = session.query(Product).all()
for p in products:
    print(f"{p.name}: ${p.price}")
```

Challenge Code:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
```

```
from sqlalchemy.orm import sessionmaker

Base = declarative_base()

class Product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    price = Column(Integer, nullable=False)

engine = create_engine("sqlite:///:memory:")
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()

# Add a Product named "Laptop" with price 999 to the session and commit
# your code here

# Query all products and print each name and price
# your code here
```

04.0 What You Know Now 🧠

Learning Objectives:

- Demonstrate understanding of what an ORM is and what problem it solves
- Identify the mapping between Python ORM concepts and their SQL equivalents
- Recognize correct relationship declarations for 1:1, 1:n, and n:m cases
- Identify session operations and their effects (commit, rollback)
- Identify the N+1 problem and its solution

Question Format: Multiple choice (9 questions)

Questions:

1. What problem does an ORM solve? a) It replaces the need to learn SQL b) It automatically creates a user interface for your database c) It maps Python objects to database tables so you don't have to convert between them manually ☑ d) It speeds up SQL queries by caching them

2. A Python class in SQLAlchemy corresponds to which SQL concept? a) A SQL query b) A database column c) A database table ☑ d) A database index

3. You define `username = Column(String(80))` on a model. What does this represent in the database? a) A `VARCHAR(80)` column ☑ b) A `TEXT` column with a max length hint c) A Python string stored as JSON d) An index on the username field

4. You have a `Post` model that belongs to a `User`. Where should the `ForeignKey("users.id")` column be placed? a) On the `User` model b) On the `Post` model ☑ c) On both models d) On a separate association table

5. You want `user.posts` to return a list of all posts by that user. What do you add to the `User` model? a) `posts = ForeignKey("posts.id")` b) `posts = relationship("Post", back_populates="author")` ☑ c) `posts = Column(List, ForeignKey("posts.id"))` d) `posts = JOIN("Post", on="user_id")`

6. What is needed to declare a many-to-many relationship between `Student` and `Course`? a) A `ForeignKey` on both the `Student` and `Course` models b) A `relationship()` on `Student` pointing to `Course` with no foreign key c) An association table with two foreign key columns, referenced via `secondary=` ☑ d) Inheriting `Course` from `Student`

7. You call `session.add(user)` but NOT `session.commit()`. What happens? a) The user is saved to the database immediately b) The user is staged for insertion but not yet saved to the database ☑ c) An error is raised because commit must always follow add d) The session auto-commits at the end of the Python script

8. Your code loads 50 orders and then accesses `order.customer` in a loop. What problem does this cause? a) A `RelationshipError` because customer is not loaded b) 51 database queries — one for the orders and one per customer ☑ c) The ORM automatically batches all 50 customer queries into one d) No problem — lazy loading is always efficient

9. Which code pattern correctly handles a failed commit? a) `session.add(obj); session.force_commit()` b) `try: session.commit() except: pass` c) `try: session.commit() except: session.rollback(); raise` ☑ d) `session.commit(safe=True)`

Evaluation Criteria:

- 9/9: Full mastery of ORM concepts, relationship types, and session usage
- 7–8/9: Strong understanding with minor gaps
- 5–6/9: Core concepts clear; review relationships and session lifecycle
- Below 5: Re-read sections 01–03 before continuing

Score Interpretation: A score of 7 or above indicates readiness for the next learnpack on database migrations with FastAPI (Alembic), which builds directly on model definition skills.

05.0 Outro

Content Outline:

- Course recap: what students accomplished
  - Understood what an ORM is and why it exists as a translator between Python and SQL
  - Mapped the correspondence: class → table, attribute → column, instance → row
  - Defined SQLAlchemy models with typed columns and primary keys
  - Declared 1:1, 1:n, and n:m relationships using ForeignKey, relationship(), and association tables
  - Used sessions to add, commit, rollback, and query records
  - Identified and resolved the N+1 performance problem

- Connection to bigger picture: the SQLAlchemy model syntax you wrote here is the exact same syntax used in real FastAPI applications — every `Base`, `Column`, and `relationship()` you'll see in production follows this same pattern
- Next steps: the next learnpack covers database migrations with FastAPI — how to use Alembic to version-control your schema and apply changes to the database safely
- Resources:
    - SQLAlchemy official docs — ORM Tutorial: https://docs.sqlalchemy.org/en/14/orm/tutorial.html
    - SQLAlchemy relationship patterns: https://docs.sqlalchemy.org/en/14/orm/relationships.html
- Encouragement: You've connected two worlds that were separate until now. Python objects and database tables are no longer different things — they're the same thing seen from two angles. That's a powerful perspective to carry forward.

Note: This is conclusion only — no theory, exercises, or assessment.
